CMSC – 35200
Deep Learning Systems

Rick Stevens

Learning Systems 2026:
Lecture 1 – Introduction



Crescat scientia; vita excolatur

Rick Stevens
Argonne National Laboratory
The University of Chicago

The 2026 course

Understand the models and the systems that make them useful.

Build with coding agents, then verify the result independently.

Measure accuracy, latency, memory, concurrency and cost.

Use scientific problems to test the limits of scalable cognition.

People and stuff

- Instructor:

  Rick Stevens rstevens@uchicago.edu (Crerar 307)

- Phd Students that are helping!

  Sep and Bahar

- Still working out plan for office hours etc.
  - Send me email for anything you need appointment, questions etc.
- We will be setting up a github repo for the class
  - You will get an email invite to join the repo

The State of AI — Late September 2026

Five forces reshaping the field at once: **frontier models, compute & datacenters, policy, enterprise adoption, and the money/risk debate.**

Frontier | **Datacenters** | **Policy** | **Adoption** | **Money & Risk**

Briefing compiled from news of the preceding weeks · one dimension per slide

Three flagships in three days — the cadence is now weekly

Sept 1–3, 2026: Anthropic, Google and OpenAI shipped frontier models on three consecutive days. Release cycles have compressed from ~yearly to near-continuous.

Sep 1	**Anthropic — Claude Fable 5.1** 1M ctx · premium autonomous work
Sep 2	**Google — Gemini 3.8 Flash** 1M ctx · fast, cheap ($0.75/$3.75), top agentic
Sep 3	**OpenAI — GPT-6 Astra** flagship reasoning; Sol/Terra/Luna value tiers below
Sep 22	**Anthropic — Claude Opus 5.5** new Claude Code default for complex coding/agents
Sep 2026	**Meta — Muse Spark 1.3** pivot to proprietary closed-weight flagship

Takeaway: context windows standardizing at 1M+, prices falling fast, and the frontier is now a crowded pack — differentiation is shifting to agentic reliability, not raw IQ.

Sources: *TeamDay AI model comparison; AI Release Tracker; DeepMind/Anthropic/OpenAI announcements (Sep 2026)*

A ~$750B/yr buildout — the largest since the interstates

~$750B
Combined 2026 capex of the largest datacenter firms (BNEF)

$690B+
Committed US buildout across 74 projects / 28 states in 2026

23+ GW
Datacenter IT capacity under construction right now

60%
Share of ALL US electrical load growth through 2030

Stargate (OpenAI · Oracle · SoftBank)

- $500B / 10 GW commitment; Abilene TX flagship
- ~300 MW live, 4 of 8 buildings operational
- 450k+ NVIDIA GB200 GPUs → Vera Rubin next
- New sites: Michigan ($45B), Jupiter NM, others
- **Altman: ~$1.4T of commitments over 8 years**

The power & grid squeeze

- 86 GW new US utility-scale power planned in 2026 (record)
- **PJM capacity auction: $2.2B → $14.7B (+568% YoY)**
- Microsoft restarted Three Mile Island for Azure
- Meta Hyperion LA: $27B / 5 GW
- Ratepayers absorbing billions across 7+ states

Takeaway: compute is now a physical-infrastructure & energy story. Power availability — not chips alone — is the binding constraint.

Sources: *BNEF; Value Add VC buildout tracker; OpenAI/Stargate releases; Forbes; IntuitionLabs (2026)*

Three regimes pulling in different directions

UNITED STATES

Deregulate + preempt

- Dec 2025 EO 14365: DOJ 'AI Litigation Task Force' to challenge state laws
- Mar 2026: WH framework urges Congress to preempt state AI law
- Preemption NOT settled — Congress declined twice; state laws still enforceable
- Patchwork: CO, CA, TX, UT, IL laws active now

EUROPEAN UNION

Binding, risk-tiered

- EU AI Act in main application phase in 2026
- Aug 2026: transparency duties (AI-content labeling, chatbot disclosure)
- Four risk tiers; some high-risk deadlines deferred (provisional)
- Sets the global high-water mark for compliance

CHINA

Targeted + tech-sovereignty

- Rules by use case (algorithms, deep synthesis, anthropomorphic AI 2026)
- Regulators discourage NVIDIA H20 purchases; push domestic chips
- DeepSeek V3.1 etc. — efficient open MoE models
- Security framing intertwined with self-reliance drive

Takeaway: companies face a genuine three-way split — deregulatory US (but contested), binding EU, use-case-targeted China. Comply with the strictest reachable regime.

Sources: *Transcend.io; Collibra; Brookings; WhiteHouse.gov EO 14365; BD Emerson (2026)*

Everyone's using it — few are running it in production

88%
of enterprises use AI in some form

39%
report measurable enterprise EBIT impact

31%
have ≥1 AI agent running in production

88%
of agent pilots never reach production

The pilot-to-production gap

- 80% of apps shipped in Q1'26 embed an agent (was 33% in '24)
- Only 16% of 'AI deployments' are true agents; rest are workflows
- 62% experimenting, 23% scaling in even one function
- **Median positive ROI: ~8 months to production**

Where it's landing

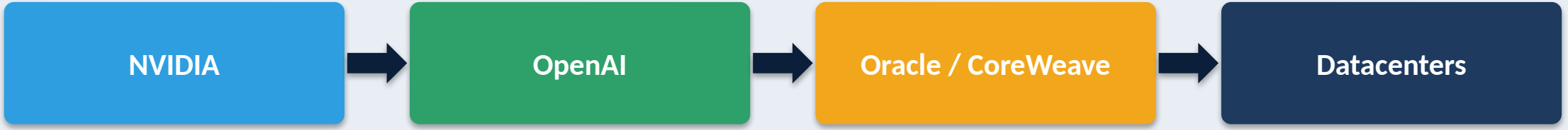
- Leaders: banking & insurance (47% in production)
- Laggards: healthcare 18%, government 14%
- **81% delayed an AI initiative — data governance, not tech**
- Gartner: 40% of enterprise apps to have task agents by end-2026

Takeaway: adoption is near-universal but shallow. The 2026 story is the hard crossing from tools to embedded, governed processes.

Sources: *McKinsey; Stanford HAI; Gartner; S&P Global; Glean; Prefactor; DigitalApplied (2026)*

The trillion-dollar bet — and the bubble question

Record capex is increasingly funded by debt and by 'circular' deals among the same handful of players. Whether that's healthy vendor finance or a fragile loop is now the central macro debate.



$100B NVIDIA→OpenAI · $300B OpenAI→Oracle · AMD + xAI + CoreWeave equity+chip deals — money circulating among backers

The case it's fine

- Mostly one-way revenue (OpenAI pays NVIDIA for chips)
- Looks like classic vendor finance / diversification
- NVIDIA stock showed no 'pop' — not obviously inflated
- Real demand: Google Cloud +63% YoY, $460B backlog

The case to worry

- MIT: ~95% of GenAI pilots show no financial return
- OpenAI partners amassed ~$100B debt; profitability unclear
- Leverage makes the system more crash-vulnerable
- 'Debt refi wall' + rogue-agent security incidents

Bottom line: capability and capital are both compounding faster than proven enterprise returns. The gap between the two is the risk to watch into 2027.

Sources: *Noahpinion; Bloomberg; FT; MIT/Fortune; Wikipedia 'AI bubble'; BNEF (2026)*

An OpenAI-compatible chat endpoint serving **Qwen3.8–27B** with full 262K context, reasoning, and tool calling. Load-balanced across 8 GPU instances. Reachable over the public internet — no VPN or special network required.

Connection details

Field	Value
Base URL	`http://209.182.219.81:4010/v1`
Model name	`qwen3.8-27b`
API key	`DLS-2026`
Protocol	OpenAI-compatible (`/v1/chat/completions`, `/v1/models`)
Auth header	`Authorization: Bearer DLS-2026`

Quick test (curl)

```
curl -s http://209.182.219.81:4010/v1/chat/completions \
  -H "Authorization: Bearer DLS-2026" \
  -H "Content-Type: application/json" \
  -d '{
    "model": "qwen3.8-27b",
    "messages": [{"role": "user", "content": "Explain a hash table in two sentences."}],
    "max_tokens": 600
  }'
```

CODEX CONFIGURATION

API

Connect Codex to a third-party endpoint

Provider selection, authentication, and validation

Choose the smallest configuration change that fits

OpenAI-compatible proxy

Keep the built-in OpenAI provider and override only its base URL.

```
openai_base_url =
  "https://gateway.example.com/v1"
```

Distinct provider

Define a named provider when authentication, headers, query parameters, or wire behavior differ.

```
model_provider = "acme"

[model_providers.acme]
base_url = "https://api.acme.ai/v1"
```

Provider routing belongs in ~/.codex/config.toml — project config cannot override it.

A custom provider needs four decisions

```
model = "YOUR_MODEL_ID"
model_provider = "acme"

[model_providers.acme]
name = "Acme Gateway"
base_url = "https://api.acme.ai/v1"
env_key = "ACME_API_KEY"
wire_api = "responses"

# Optional provider-specific settings
http_headers = { "X-Tenant" = "engineering" }
query_params = { region = "us-central" }
```

1

Model ID

Use the identifier exposed by the endpoint.

2

Base URL

Include the provider's required API prefix.

3

Authentication

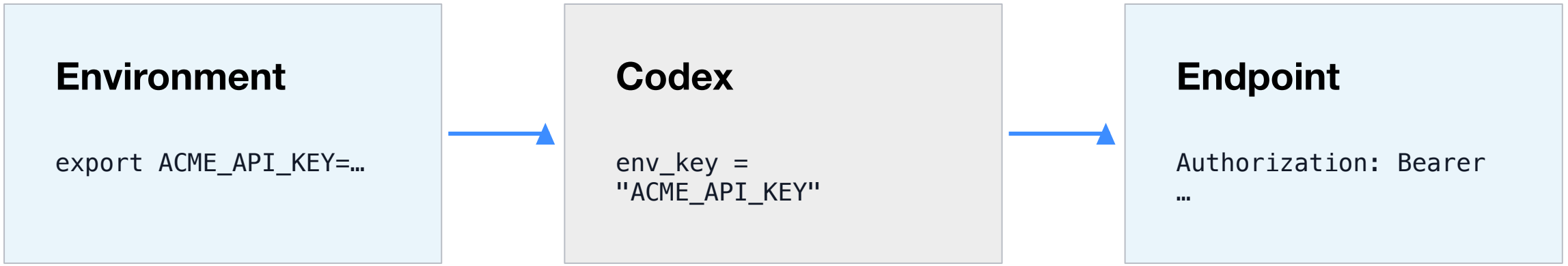
Reference an environment variable; do not paste a secret.

4

Wire API

Use responses only when the endpoint implements compatible behavior.

Keep credentials outside the configuration file



Alternative: configure [model_providers.<id>.auth] to run a credential helper that prints a bearer token.

Do not combine command-backed auth with env_key, experimental_bearer_token, or requires_openai_auth.

Validate the connection in three passes

01

Confirm config

Provider ID matches
Model ID exists
Base URL includes prefix

02

Confirm auth

Variable is exported
Token reaches the endpoint
No secrets in TOML

03

Run a small request

Start Codex
Send a simple prompt
Inspect endpoint logs

401/403 → auth • 404 → base path or model • 400 → incompatible request schema • stream stalls → proxy buffering/timeouts



The pi approach to CLI agents

primitives, not features

```
$ npm install -g @earendil-works/pi-coding-agent
```

pi.dev · open source (MIT) · a deep dive for people who already live in coding agents

ORIGINS

Why write another CLI agent?

pi started as Mario Zechner’s reaction to what mainstream harnesses had become — excellent models wrapped in machinery you can’t see into or reshape. Today it’s developed in the open by Earendil, shipping weekly.

+

Feature accretion

Sub-agents, plan modes, todo widgets, MCP menus — capabilities stacked on by default, whether or not they help the model do the work in front of it.

?

Invisible context

Harnesses “inject stuff behind your back that isn’t even surfaced in the UI.” You can’t engineer a context window you can’t inspect.

×

Closed workings

Session state in opaque formats, system prompts that change under you between releases, cores you can’t rewire or embed.

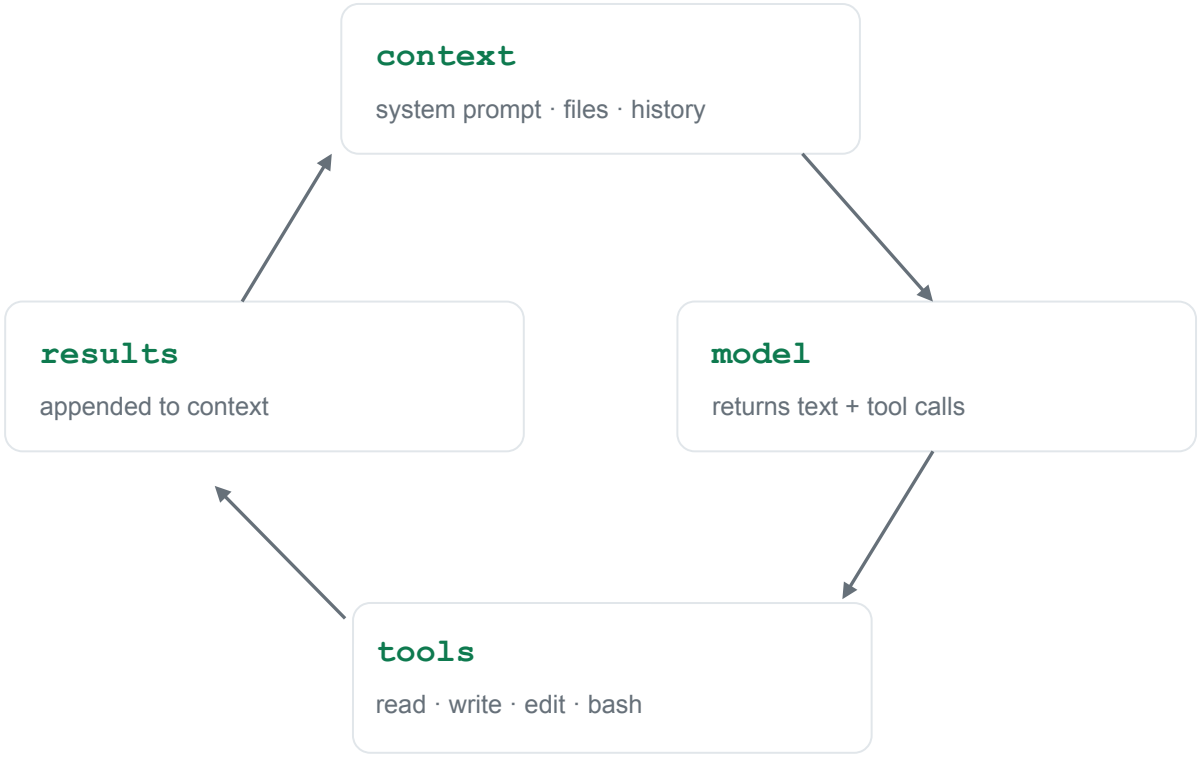
The founding rule: ***“if I don’t need it, it won’t be built.”***

A coding agent is a small loop

Strip any CLI agent to its skeleton and you find the same thing: send context to a model, run the tool calls it returns, append the results, repeat until it stops asking.

Everything else — planning modes, tool menus, orchestration — is UI and context management layered on top.

pi's wager: capability lives in the model. The harness should stay small, legible, and yours — and get out of the way.




```
while (true) { llm(ctx) → tools() → ctx += results }
```

PHILOSOPHY

The pi approach, in five commitments

$

Tiny core

Four tools and a ~200-word system prompt. Nothing ships on by default that isn’t essential.

#

Context is yours

Every token in the window is visible, deliberate, and put there by you — never injected silently.

>

Files over features

Plans, todos, and knowledge live as markdown files — durable, diffable, editor-agnostic.

+

Extend, don’t configure

Missing features aren’t settings to toggle — they’re TypeScript you (or your agent) write.

~

Transparent by design

JSONL sessions, an inspectable prompt, MIT source, and an SDK that exposes the same core.

“Adapt pi to your workflows, not the other way around.” — pi.dev

SMALL CORE

A core you can read in one sitting

~200

WORDS OF SYSTEM PROMPT

The entire behavioral instruction pi gives the model.

< 1k

TOKENS, PROMPT + TOOLS

System prompt and all tool definitions combined.

4

TOOLS ON BY DEFAULT

read · write · edit · bash — the whole default toolbox.

0

HIDDEN INJECTIONS

Nothing enters the context window behind your back.

Why so small?

Frontier models arrive “RL-trained up the wazoo” for agentic coding. They don’t need a ten-thousand-token essay on how to behave — they need a clean window with the right facts in it. Verbose prompts mostly compensate for weak context discipline.

“The shortest system prompt of any agent that I’m aware of.”

— Armin Ronacher, on switching to pi

The whole system prompt, verbatim

as printed in the design write-up

```
You are an expert coding assistant. You help users with coding tasks by reading files,
executing commands, editing code, and writing new files.

Available tools:
- read: Read file contents
- bash: Execute bash commands
- edit: Make surgical edits to files
- write: Create or overwrite files

Guidelines:
- Use bash for file operations like ls, grep, find
- Use read to examine files before editing
- Use edit for precise changes (old text must match exactly)
- Use write only for new files or complete rewrites
- When summarizing your actions, output plain text directly - do NOT use cat or bash
  to display what you did
- Be concise in your responses
- Show file paths clearly when working with files

Documentation:
- Your own documentation (including custom model setup and theme creation) is at:
  /path/to/README.md
- Read it when users ask about features, configuration, or setup, and especially if the
  user asks you to add a custom model or provider, or create a custom theme.
```

~200 words

That’s all of it. With the four tool definitions added, the model starts under 1,000 tokens.

+ AGENTS.md

Your instructions are appended below it — global first, then per-project.

SYSTEM.md

Don’t like it? A SYSTEM.md replaces the whole thing. The core prompt is yours too.

docs on demand

Note the last section: even pi’s own feature docs load only when asked about — progressive disclosure, applied to itself.

TOOLS

The whole toolbox

$ read

Files and images. Returns up to 2,000 lines by default; the model asks for more if it needs it.

$ write

Create or overwrite a file. No partial magic — what you see is what lands on disk.

$ edit

Surgical replacement. The old text must match exactly, which keeps edits honest.

$ bash

Synchronous shell. Output goes straight back into context — no detached jobs to lose track of.

Optional, off by default: grep · find · ls — kept aside because the model usually does better composing bash itself.

Every other capability the model ever gets — browsers, deploy scripts, tickets, databases — arrives as **CLI tools it discovers, skills it reads, or extensions you load**. The core toolbox never grows.

SUBTRACTIONS

What pi refuses to ship by default

FEATURE	WHY IT'S OUT	WHAT PI EXPECTS INSTEAD
MCP client	Resident tool schemas tax every single request, used or not.	→ CLI tools with READMEs, read on demand.
Plan mode	Plans as ephemeral UI state die with the session.	→ The plan is a markdown file in the repo.
Todo widget	Built-in lists "confuse models more than they help."	→ A checklist file the agent keeps updated.
Sub-agents	Opaque delegation; context transfer is lossy both ways.	→ Branch the session — or add them via extension.
Background bash	Detached processes the transcript can't account for.	→ tmux — you and the agent share the terminal.

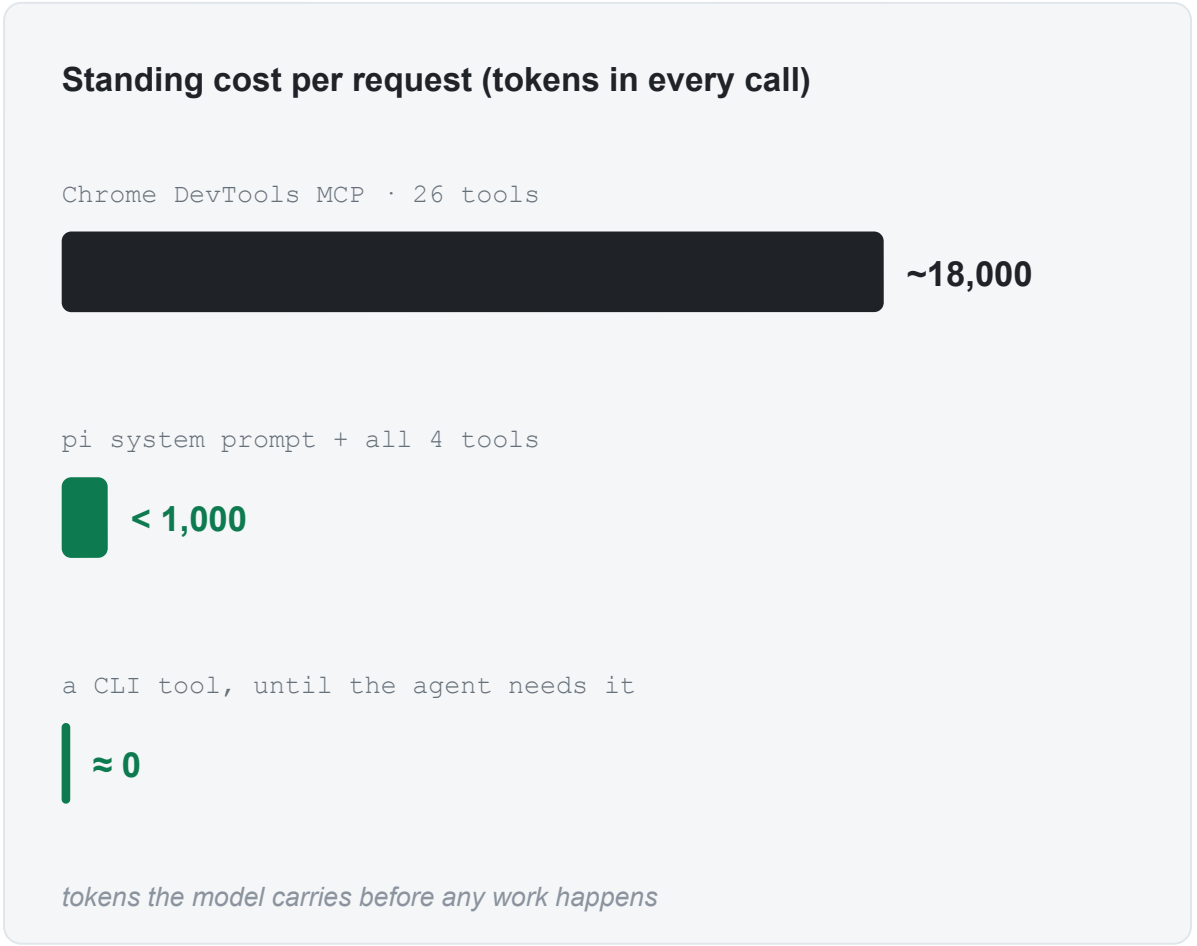
Every one of these exists as an optional extension. Subtraction from the core is not prohibition — it's a default.

Why no MCP? Count the tokens

Connect one popular MCP server — Chrome DevTools — and 26 tool definitions ride along in every request: roughly 18,000 tokens, 7–9% of the window, spent before the first prompt.

pi's alternative: package capability as CLI tools with a README. The agent runs them like any program and reads the docs only when the task calls for it — progressive disclosure instead of standing overhead.

Reference set: github.com/badlogic/agent-tools — browser control, scraping, and more, all plain CLIs.



FILES OVER FEATURES

Plans and todos are just files

Plan modes and todo widgets live inside one harness's UI state — invisible to your editor, gone when the session ends, unreadable by any other tool.

In pi you ask the model to write PLAN.md, edit it in anything, and have the agent keep it current as it works. Files survive sessions, travel with the repo, diff in git, and any agent — or colleague — can pick them up.

> *"To-do lists generally confuse models more than they help."*
> — the pi position, from the design write-up

PLAN.md

```
$ cat PLAN.md

# auth refactor — plan

- [x] map the current session flow
- [x] introduce TokenStore interface
- [ ] migrate login handlers
- [ ] delete legacy cookie path

## notes
keep each PR under 400 lines;
run the e2e suite before merging.

# edited by you, updated by the agent —
# and readable by every tool you own.
```

Sub-agents: opacity you pay for

×

Zero visibility

“You have zero visibility into what that sub-agent does.” The transcript stops being a complete record of the work.

≠

Lossy handoff

Context in and results out are summaries of summaries. The parent reasons about work it never actually saw.

!

A planning smell

Reaching for a sub-agent mid-session usually means the task wasn’t decomposed up front — the fix is planning, not machinery.

Quotes: Mario Zechner, pi design write-up

What pi expects instead

→ **Branch the session**

Explore the risky path on a branch of the session tree; keep or abandon it with a summary.

→ **Decompose into files**

Split big work into planned chunks — fresh sessions pick up PLAN.md with clean windows.

→ **Opt in via extension**

Genuinely need fan-out? Sub-agents ship among pi’s 50+ example extensions — visible in the session like everything else.

You can name every token in the window

system prompt — < 1k tok

~200 words — the whole thing

AGENTS.md — yours

your project instructions, from the repo

SYSTEM.md — optional

optional full prompt override

skill index — tiny

names + one-line descriptions only

conversation — the bulk

your messages, model turns, tool results

…and nothing else gets in.

> *“Exactly controlling what goes into the model’s context yields better outputs, especially when it’s writing code.”*

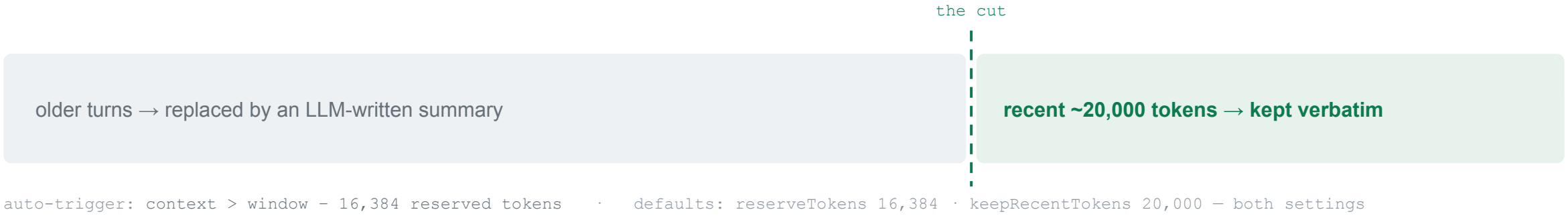
Everything in the stack on the left is a file you own or a message you sent. Nothing is rewritten between turns, nothing appears on model switch, no “helpful” reminders are spliced into your conversation.

Want dynamic injection — repo state, tickets, telemetry? You can have it. But you write the extension that does it, so you always know it’s there.

LONG SESSIONS

Compaction without amnesia

When the window fills, pi summarizes the past instead of truncating it — and it does the bookkeeping that naive compaction loses.



Split turns

If the cut lands mid-turn, pi writes two coordinated summaries — prior context and the turn's early half — so nothing falls in the crack.

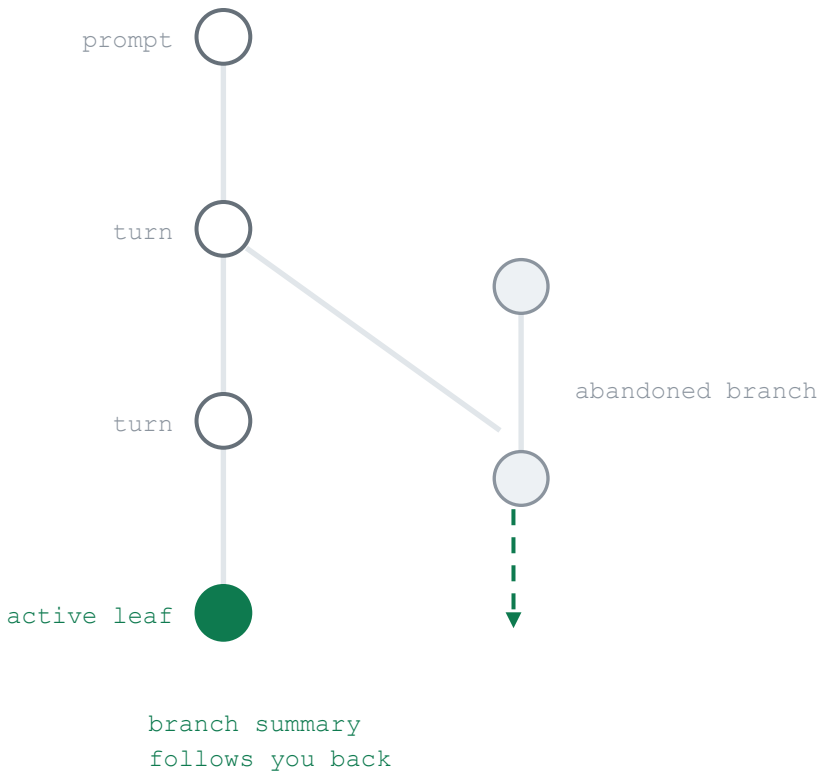
Cumulative file tracking

Every file read and edit is carried through summaries of summaries. After three compactions, pi still knows what it touched.

/compact "keep the SQL"

Compaction takes instructions. Steer what the summary preserves instead of hoping the summarizer guesses right.

Sessions are trees, not scrollback



Every entry is a JSONL record with an id and parentId, stored in ~/.pi/agent/sessions. Your position is just the active leaf — so “undo”, “try again differently”, and “what if” are all the same cheap operation: move.

/tree	navigate and fold the whole history; pick any earlier message, edit it, and branch from there.
branch summaries	leaving a branch offers to summarize it — the context of the road not taken follows you.
/fork · /clone · /resume	new file from an earlier prompt · duplicate the active branch · reopen anything later.
/export · /share	self-contained HTML, or a private GitHub gist link for a colleague.

Transparent format, no lock-in: any script can parse a session.